

PROJECT REPORT: PLANET HUNTER

An Interactive Sci-Fi Stellar Classification Engine Using Machine Learning

1. Project Abstract

Planet Hunter is an interactive, science-fiction inspired web application that integrates Supervised Machine Learning with user experience design to classify celestial bodies. Serving as the tactical core intelligence platform for a deep-space scouting vessel, the system dynamically analyzes real-world physical properties of stars—such as surface temperature, luminosity, dimensional radius, and absolute magnitude. The primary objective of the system is to automate stellar taxonomy and successfully isolate stable, Sun-like stars capable of serving as humanity's next cosmic sanctuary (*Nova Terra*). The project bridges data science with responsive web design to create an educational, immersive narrative experience.

2. Introduction & Problem Statement

In modern astrophysics, cataloging millions of stars detected by deep-space telescopes is an immense task that cannot be handled manually. Traditional sorting using the Hertzsprung-Russell (H-R) Diagram relies heavily on manual plotting of temperature against intrinsic brightness.

Problem Solved:

This project solves the problem of manual identification by deploying an automated AI Agent capable of instant stellar processing. Embedded into a creative science-fiction theme, it removes technical barriers for users, allowing them to instantly discover star types (such as Red Dwarfs, White

Dwarfs, Main Sequence stars, Giants, or Supergiants) via a simple graphical user interface, making astronomical data processing highly accessible and engaging.

3. System Architecture & Tech Stack

The application is built using a modern, lightweight decoupled architecture where the front-end interface communicates seamlessly with an analytical Python back-end engine.

- **Backend Framework: Flask (Python)** — Handles master server orchestration, API routing, and hosts the endpoint (/predict) responsible for structural data transfer.
- **Data Processing Engine: Pandas & NumPy** — Utilized for internal data handling, vector processing, parsing raw storage files, and feature isolating.
- **Machine Learning Library: Scikit-Learn** — Powers the prediction architecture through its core math model logic.
- **Frontend UI Layer: Semantic HTML5, Custom Glassmorphic CSS3, and Vanilla JavaScript** — Manages the immersive responsive dashboard controls, captures range slider inputs asynchronously, and plays navigational audio feedback.

4. Machine Learning Model & Logic

The foundational brain of the AI Agent is built upon a **Decision Tree Classifier**, a popular supervised machine learning algorithm.

Dynamic On-the-Fly Training:

Instead of saving a pre-trained model to a static serialized file (like .joblib or .pkl), which often crashes due to local system version mismatches, this

project utilizes **runtime memory compilation**. The moment the Flask server boots up, Pandas reads the raw dataset (star_datatype.csv) containing historical NASA stellar observations. The features are immediately mapped, and the Decision Tree is instantiated directly inside the computer's volatile memory (RAM).

Algorithmic Flow:

The Decision Tree mimics real-world physics by making binary conditional check splits at structural root and internal nodes:

1. **Node Split 1:** Evaluates the dimensional scale (*Is Radius $\leq$ Threshold?*). If True, it moves toward dense dwarf regions; if False, it moves toward expanded giants.
2. **Node Split 2:** Evaluates surface heat indices (*Temperature*) and brightness values (*Luminosity & Absolute Magnitude*).
3. **Leaf Node:** Pinpoints the exact target classification matching the user's inputs.

5. Dataset Features Evaluated

The model initializes its training weights using four fundamental scientific features:

1. **Temperature (Kelvin):** Controls the heat metric of the stellar core and determines peak color emission (e.g., lower temps for Red Dwarfs, extreme temps for Blue Giants).
2. **Luminosity ($L/L_{\odot}$):** Measures the relative total energy output of the star compared directly to our Sun.
3. **Radius ($R/R_{\odot}$):** Measures the structural spatial volume and physical dimension relative to our Sun.

4. **Absolute Magnitude (M_v):** Quantifies the absolute intrinsic visual brightness of the celestial body independent of its distance from a telescope.

6. The Narrative Agent Interface (Marry & Rocky)

To fulfill professional AI design requirements, the system operates as a **Model-Based Reflex Agent** packaged inside an immersive storyline:

- **Marry (The Ships Voice Core):** Serves as the high-level computational parsing interface, translating raw machine learning classification strings into adaptive feedback.
- **Rocky (The Support Bot):** Provides critical contextual alerts depending on the classification safety rating (e.g., panicking and warning the crew if a volatile, explosive Supergiant is detected, or celebrating if a stable Main Sequence star matches the parameters for *Nova Terra*).

7. Future Enhancements

- **Persistent Database Storage:** Integrating an SQLite3 or PostgreSQL database to save scanned star systems discovered by players.
- **Advanced Models:** Incorporating Random Forest Classifiers or Support Vector Machines (SVM) to show comparative accuracy metrics on the dashboard.
- **Interactive Visual Graphs:** Utilizing libraries like Chart.js or Plotly to render a dynamic Hertzsprung-Russell Diagram plotting the user's custom star live on the screen.

8. Conclusion

Planet Hunter successfully demonstrates how machine learning models can be utilized outside standard command-line tools to create fully operational, deployment-ready web apps. By coupling a Python-based Scikit-Learn engine with an optimized Flask interface, the project provides reliable, fast classification of stellar bodies while maintaining an exceptional, responsive user experience